

CP 期中练习

2025-12-29

Aksel Shen

20234001053@m.scnu.edu.cn

South China Normal University

1. (解答题, 每题 10 分)

考虑字母表 $\{a, b\}$ 下的正则表达式

$$(a|b)^* ab(a|b)^*$$

(1) 请给出正则表达式对应的 NFA。

(2) 请给出上述 NFA 对应的 DFA 并最小化 DFA 的状态。

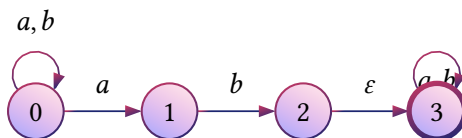
解

(1) 正则表达式对应的 NFA

将正则表达式 $(a|b)^* ab(a|b)^*$ 分解为若干部分:

- $(a|b)^*$: 可以匹配任意个 a 或 b
- a : 匹配一个 a
- b : 匹配一个 b
- $(a|b)^*$: 可以匹配任意个 a 或 b

构造 NFA 如下:



(2) NFA 转 DFA 并最小化

使用子集构造法将 NFA 转换为 DFA。

首先计算 ϵ -closure:

- $\epsilon\text{-closure}(\{0\}) = \{0\}$
- $\epsilon\text{-closure}(\{1\}) = \{1\}$
- $\epsilon\text{-closure}(\{2\}) = \{2, 3\}$
- $\epsilon\text{-closure}(\{3\}) = \{3\}$

构造 DFA 状态转换:

从初始状态 $\{0\}$ 开始:

- $\delta(\{0\}, a) = \epsilon\text{-closure}(\{0, 1\}) = \{0, 1\}$
- $\delta(\{0\}, b) = \epsilon\text{-closure}(\{0\}) = \{0\}$

从状态 $\{0, 1\}$:

- $\delta(\{0, 1\}, a) = \epsilon\text{-closure}(\{0, 1\}) = \{0, 1\}$
- $\delta(\{0, 1\}, b) = \epsilon\text{-closure}(\{0, 2, 3\}) = \{0, 2, 3\}$

从状态 $\{0, 2, 3\}$ (包含终态):

- $\delta(\{0, 2, 3\}, a) = \epsilon\text{-closure}(\{0, 1, 3\}) = \{0, 1, 3\}$
- $\delta(\{0, 2, 3\}, b) = \epsilon\text{-closure}(\{0, 3\}) = \{0, 3\}$

从状态 $\{0, 1, 3\}$ (包含终态):

- $\delta(\{0, 1, 3\}, a) = \varepsilon\text{-closure}(\{0, 1, 3\}) = \{0, 1, 3\}$
- $\delta(\{0, 1, 3\}, b) = \varepsilon\text{-closure}(\{0, 2, 3\}) = \{0, 2, 3\}$

从状态 $\{0, 3\}$ (包含终态):

- $\delta(\{0, 3\}, a) = \varepsilon\text{-closure}(\{0, 1, 3\}) = \{0, 1, 3\}$
- $\delta(\{0, 3\}, b) = \varepsilon\text{-closure}(\{0, 3\}) = \{0, 3\}$

DFA 状态表:

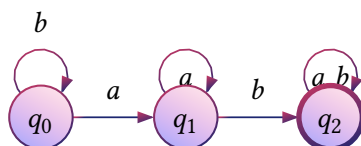
State	a	b
$q_0 = \{0\}$	q_1	q_0
$q_1 = \{0, 1\}$	q_1	q_2
$q_2 = \{0, 2, 3\}^*$	q_3	q_4
$q_3 = \{0, 1, 3\}^*$	q_3	q_2
$q_4 = \{0, 3\}^*$	q_3	q_4

最小化 DFA:

初始划分: 非终态 $\{q_0, q_1\}$, 终态 $\{q_2, q_3, q_4\}$ 。

- 对于 $\{q_0, q_1\}$:
 - $\delta(q_0, a) = q_1, \delta(q_1, a) = q_1$ (同属非终态)
 - $\delta(q_0, b) = q_0, \delta(q_1, b) = q_2$ (q_0 属非终态, q_2 属终态)
 - 因此 q_0 和 q_1 需要分离
- 对于 $\{q_2, q_3, q_4\}$:
 - $\delta(q_2, a) = q_3, \delta(q_3, a) = q_3, \delta(q_4, a) = q_3$ (均属终态)
 - $\delta(q_2, b) = q_4, \delta(q_3, b) = q_2, \delta(q_4, b) = q_4$ (均属终态)
 - 所有转换都指向同一组, 因此暂时不分离

继续细分, 最终得到最小化 DFA:



2. (解答题, 每题 10 分)

考虑有如下的文法 $G[S]$:

$$S \rightarrow aSbS$$

$$S \rightarrow aS$$

$$S \rightarrow c$$

(1) 请画出输入串 $acbac$ 对应的分析树。

(2) 请说明 G 是一个二义文法。

解

(1) 输入串 $acbac$ 的分析树

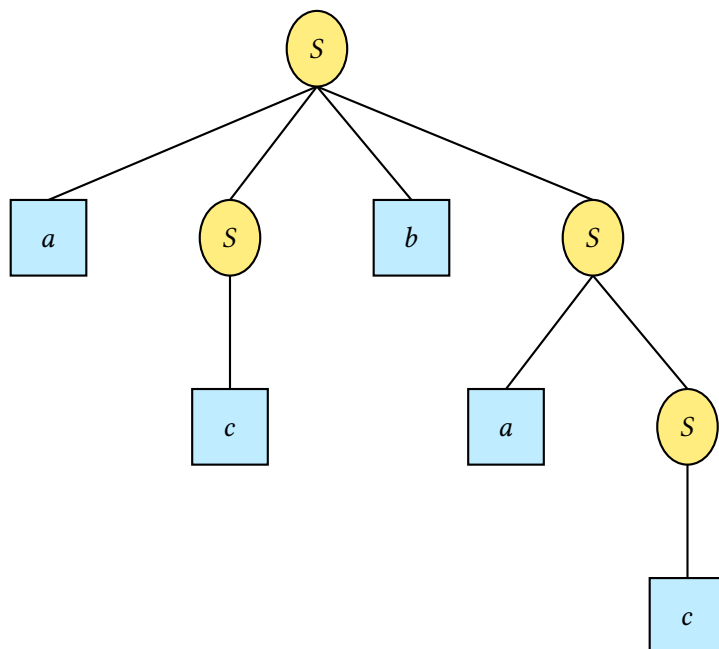


图 3 推导: $S \Rightarrow aSbS \Rightarrow acbS \Rightarrow acbaS \Rightarrow acbac$

(2) 证明 G 是二义文法

对于输入串 $aacbc$, 存在两棵不同的分析树:

分析树 1: 使用推导 $S \Rightarrow aSbS \Rightarrow aaSbS \Rightarrow aacbS \Rightarrow aacbc$

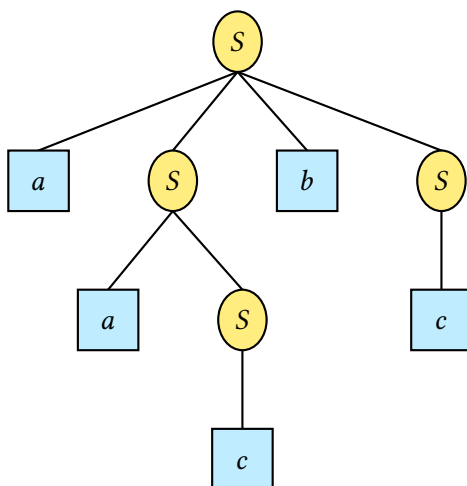


图 4 推导 1: $S \Rightarrow aSbS \Rightarrow aaSbS \Rightarrow aacbS \Rightarrow aacbc$

分析树 2: 使用推导 $S \Rightarrow aS \Rightarrow aaSbS \Rightarrow aacbS \Rightarrow aacbc$

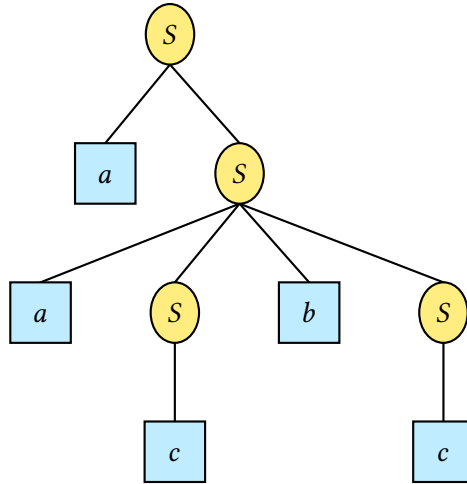


图 5 推导 2: $S \Rightarrow aS \Rightarrow aaSbS \Rightarrow aacbS \Rightarrow aacbc$

因为同一个句子存在两棵不同的分析树，所以文法 G 是二义的。

3. (解答题, 每题 15 分)

考虑有如下的文法 $G[S]$:

$$S \rightarrow aBc \mid bAB$$

$$A \rightarrow aBb \mid b$$

$$B \rightarrow b \mid \varepsilon$$

(1) 请构造文法 G 的 LL(1) 分析表。

(2) 请给出符号串 $baabbb$ 的分析过程，并说明它是否是一个合法的输入。

解

(1) 构造 LL(1) 分析表

首先计算 FIRST 和 FOLLOW 集合:

FIRST 集合:

- $\text{FIRST}(S) = \{a, b\}$
- $\text{FIRST}(A) = \{a, b\}$
- $\text{FIRST}(B) = \{b, \varepsilon\}$

FOLLOW 集合:

- $\text{FOLLOW}(S) = \{\$ \}$
- $\text{FOLLOW}(A) = \{b\}$
- $\text{FOLLOW}(B) = \{c, b, \$ \}$

对于每个产生式:

- (1) $S \rightarrow aBc$: $\text{FIRST}(aBc) = \{a\}$, 所以在 $M[S, a]$ 中填入 $S \rightarrow aBc$
- (2) $S \rightarrow bAB$: $\text{FIRST}(bAB) = \{b\}$, 所以在 $M[S, b]$ 中填入 $S \rightarrow bAB$
- (3) $A \rightarrow aBb$: $\text{FIRST}(aBb) = \{a\}$, 所以在 $M[A, a]$ 中填入 $A \rightarrow aBb$
- (4) $A \rightarrow b$: $\text{FIRST}(b) = \{b\}$, 所以在 $M[A, b]$ 中填入 $A \rightarrow b$
- (5) $B \rightarrow b$: $\text{FIRST}(b) = \{b\}$, 所以在 $M[B, b]$ 中填入 $B \rightarrow b$

(6) $B \rightarrow \varepsilon$: 因为 $\varepsilon \in \text{FIRST}(B)$, 对于 $\text{FOLLOW}(B) = \{c, b, \$\}$ 中的每个符号, 在 $M[B, c]$ 、 $M[B, b]$ 和 $M[B, \$]$ 中填入 $B \rightarrow \varepsilon$

注意: $M[B, b]$ 中同时有 $B \rightarrow b$ 和 $B \rightarrow \varepsilon$, 存在冲突, 因此该文法不是 LL(1) 文法。

LL(1) 分析表 (带冲突):

	a	b	c	$\$$
S	$\rightarrow aBc$	$\rightarrow bAB$		
A	$\rightarrow aBb$	$\rightarrow b$		
B		$\rightarrow b$ $\rightarrow \varepsilon$	$\rightarrow \varepsilon$	$\rightarrow \varepsilon$

(2) 符号串 $baabbb$ 的分析过程

虽然文法不是 LL(1), 但我们可以尝试分析该符号串。由于在状态 $[B, b]$ 存在冲突, 我们需要做出选择。

步骤	分析栈	输入串	使用的产生式
1	$\$S$	$baabbb\$$	
2	$\$cBa$	$baabbb\$$	$S \rightarrow aBc$ (错误)

从步骤 2 可以看出, 选择 $S \rightarrow aBc$ 后无法匹配输入的 b , 因此应选择 $S \rightarrow bAB$ 。

重新分析:

步骤	分析栈	输入串	使用的产生式
1	$\$S$	$baabbb\$$	
2	$\$BAb$	$baabbb\$$	$S \rightarrow bAB$
3	$\$BA$	$aabbb\$$	匹配 b
4	$\$BbBa$	$aabbb\$$	$A \rightarrow aBb$
5	$\$BbB$	$abbb\$$	匹配 a
6	$\$BbBb$	$abbb\$$	$B \rightarrow b$ (选择 1)
7	$\$BbB$	$bbb\$$	匹配 b
8	$\$BbBb$	$bbb\$$	$B \rightarrow b$ (选择 1)
9	$\$BbB$	$bb\$$	匹配 b

此时无法继续分析。让我们在步骤 6 时选择 $B \rightarrow \varepsilon$:

步骤	分析栈	输入串	使用的产生式
1	$\$S$	$baabbb\$$	
2	$\$BAb$	$baabbb\$$	$S \rightarrow bAB$
3	$\$BA$	$aabbb\$$	匹配 b
4	$\$BbBa$	$aabbb\$$	$A \rightarrow aBb$
5	$\$BbB$	$abbb\$$	匹配 a
6	$\$Bb$	$abbb\$$	$B \rightarrow \varepsilon$
7	$\$B$	$bbb\$$	匹配 b (错误)

步骤 7 出错，因为栈顶是 B 但已匹配了 b ，输入还是 a 。

经过多次尝试，该符号串 **不是** 该文法的合法输入。

4. (证明题，每题 15 分)

考虑由正则表达式 r 所生成的语言 L ， L' 是将 L 中每个符号串反转后得到的语言：

$$L' = \{a_1 a_2 \cdots a_n \mid a_n \cdots a_2 a_1 \in L, n \geq 0\}$$

请证明， L' 也是某个正则表达式 r' 所生成的语言。

证明

我们通过对正则表达式 r 的结构进行归纳来证明。

基础情况：

- (1) 若 $r = \emptyset$ (空集)，则 $L = \emptyset$ ， $L' = \emptyset$ ， $r' = \emptyset$ 。
- (2) 若 $r = \varepsilon$ (空串)，则 $L = \{\varepsilon\}$ ， $L' = \{\varepsilon\}$ ， $r' = \varepsilon$ 。
- (3) 若 $r = a$ (单个符号)，则 $L = \{a\}$ ， $L' = \{a\}$ ， $r' = a$ 。

归纳步骤：

假设对于正则表达式 r_1 和 r_2 ，其对应的语言为 L_1 和 L_2 ，反转后的语言 $L_{1'}$ 和 $L_{2'}$ 分别由正则表达式 $r_{1'}$ 和 $r_{2'}$ 生成。

- (1) **并运算：** 若 $r = r_1 \mid r_2$ ，则 $L = L_1 \cup L_2$ 。

对于任意 $w \in L'$ ，存在 $w^R \in L$ (其中 w^R 表示 w 的反转)。

- 若 $w^R \in L_1$ ，则 $w \in L_{1'}$ ，由归纳假设知 w 可由 $r_{1'}$ 生成。
- 若 $w^R \in L_2$ ，则 $w \in L_{2'}$ ，由归纳假设知 w 可由 $r_{2'}$ 生成。

因此 $L' = L_{1'} \cup L_{2'}$ ，取 $r' = r_{1'} \mid r_{2'}$ 。

- (2) **连接运算：** 若 $r = r_1 r_2$ ，则 $L = L_1 L_2 = \{w_1 w_2 \mid w_1 \in L_1, w_2 \in L_2\}$ 。

对于任意 $w \in L'$ ，存在 $w^R \in L$ ，即 $w^R = w_1 w_2$ ，其中 $w_1 \in L_1$ ， $w_2 \in L_2$ 。

$$\text{则 } w = (w_1 w_2)^R = w_2^R w_1^R。$$

由归纳假设， $w_1^R \in L_{1'}$ 可由 $r_{1'}$ 生成， $w_2^R \in L_{2'}$ 可由 $r_{2'}$ 生成。

因此 $L' = \{w_2^R w_1^R \mid w_1 \in L_1, w_2 \in L_2\} = L_{2'} L_{1'}$ ，取 $r' = r_{2'} r_{1'}$ (注意顺序反转)。

- (3) **Kleene 闭包：** 若 $r = r_1^*$ ，则 $L = L_1^* = \{w_1 w_2 \cdots w_k \mid k \geq 0, w_i \in L_1\}$ 。

对于任意 $w \in L'$ ，存在 $w^R \in L^*$ ，即 $w^R = w_1 w_2 \cdots w_k$ ，其中每个 $w_i \in L_1$ 。

$$\text{则 } w = (w_1 w_2 \cdots w_k)^R = w_k^R \cdots w_2^R w_1^R。$$

由归纳假设，每个 $w_i^R \in L_{1'}$ 可由 $r_{1'}$ 生成。

因此 $L' = \{w_k^R \cdots w_1^R \mid k \geq 0, w_i \in L_1\} = (L_{1'})^*$ ，取 $r' = (r_{1'})^*$ 。

结论：

由数学归纳法，对于任意正则表达式 r 生成的语言 L ，其反转语言 L' 也可以由某个正则表达式 r' 生成。构造 r' 的规则如下：

- $(\emptyset)^R = \emptyset$
- $(\varepsilon)^R = \varepsilon$

- $(a)^R = a$
- $(r_1 | r_2)^R = r_1^R | r_2^R$
- $(r_1 r_2)^R = r_2^R r_1^R$
- $(r_1^*)^R = (r_1^R)^*$

5. (分析题, 每题 15 分)

考虑语言

$$L = \{a^m b^n \mid 0 \leq m \leq 2n\}$$

即 L 中的串由若干个 a 和若干个 b 组成, 并且 a 的个数不超过 b 的个数的两倍。

请写出三个文法表示 L , 并且分别是 LR(1) 文法、二义文法、非二义并且非 LR(1) 的文法。

解

(1) LR(1) 文法

$$\begin{aligned} S &\rightarrow AB \\ A &\rightarrow aAb \mid aAbb \mid \varepsilon \\ B &\rightarrow bB \mid \varepsilon \end{aligned}$$

该文法通过 A 生成满足 $m \leq 2n$ 的 $a^m b^n$ 部分, 其中:

- $A \rightarrow aAb$ 生成 a 和 b 数量相等的部分
- $A \rightarrow aAbb$ 生成 a 的数量为 b 的一半的部分
- B 生成额外的 b

该文法是 LR(1) 的, 因为它没有冲突, 可以通过 LR(1) 分析表进行分析。

实际上, 更简洁的 LR(1) 文法如下:

$$S \rightarrow aSb \mid aSbb \mid \varepsilon$$

该文法可以递归地生成符合条件的串:

- $S \rightarrow aSb$: 每次添加一个 a 和一个 b
- $S \rightarrow aSbb$: 每次添加一个 a 和两个 b
- $S \rightarrow \varepsilon$: 终止递归

(2) 二义文法

$$S \rightarrow aSb \mid SS \mid aSbb \mid \varepsilon$$

该文法是二义的, 因为对于某些串 (如 $abab$), 存在多种推导方式:

推导 1: $S \Rightarrow SS \Rightarrow aSbS \Rightarrow abS \Rightarrow abaSb \Rightarrow abab$

推导 2: $S \Rightarrow SS \Rightarrow aSbaSb \Rightarrow abaSb \Rightarrow abab$

这两种推导对应不同的分析树, 因此该文法是二义的。

(3) 非二义且非 LR(1) 的文法

$$\begin{aligned} S &\rightarrow A \\ A &\rightarrow aAb \mid aAbb \mid B \\ B &\rightarrow aBb \mid aBbb \mid \varepsilon \end{aligned}$$

该文法是非二义的, 因为每个串只有唯一的推导。但是, 由于 A 和 B 的产生式结构相似, 在 LR 分析时会产生归约/归约冲突, 因此不是 LR(1) 文法。

具体来说,当面临 $a^k b^j$ 的前缀时,分析器无法决定是使用 A 的产生式还是 B 的产生式进行归约。

6. (综合题, 每题 20 分)

某同学在使用自顶向下分析方法来处理正整数的四则运算时,写下了如下的文法:

$$\begin{aligned} E &\rightarrow E + E \\ E &\rightarrow E - E \\ E &\rightarrow E * E \\ E &\rightarrow E / E \\ E &\rightarrow (E) \\ E &\rightarrow n \end{aligned}$$

- (1) 请问如果使用上述文法进行自顶向下分析, 会存在哪些问题。
- (2) 请将上述文法改写为可以进行自顶向下分析的语法, 并且能满足先加减后乘除的要求。
- (3) 请实现改写后文法的递归下降分析程序 `TreeNode* parse(char* exp)`, 对于输入的表达式 `exp`, 返回这个表达式对应的语法树。请自行定义语法树节点 `TreeNode` 的数据结构。为了简化代码, 可以假设表达式中的数均为一位数字。

解

(1) 存在的问题

使用该文法进行自顶向下分析存在以下问题:

- (1) **左递归**: 文法中存在直接左递归 (如 $E \rightarrow E + E$), 这会导致递归下降分析器陷入无限循环。当尝试展开 E 时, 会立即再次遇到 E , 无法终止。
- (2) **二义性**: 文法是二义的, 对于同一个表达式 (如 $n + n * n$) 可以有多种不同的分析树, 无法确定运算符的优先级和结合性。
- (3) **无法确定优先级**: 文法没有区分不同运算符的优先级, 无法实现“先乘除后加减”的要求。

(2) 改写后的文法

为了支持自顶向下分析并满足先乘除后加减的要求, 将文法改写如下:

$$\begin{aligned} E &\rightarrow TE' \\ E' &\rightarrow +TE' \mid -TE' \mid \varepsilon \\ T &\rightarrow FT' \\ T' &\rightarrow *FT' \mid /FT' \mid \varepsilon \\ F &\rightarrow (E) \mid n \end{aligned}$$

该文法的特点:

- 消除了左递归, 改用右递归形式
- 通过层次结构实现了运算符优先级: E (加减) $\rightarrow T$ (乘除) $\rightarrow F$ (因子)
- E' 和 T' 用于处理同级运算符的右结合

(3) 递归下降分析程序

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>

// 定义语法树节点类型
typedef enum {
    NODE_NUM, // 数字节点
```



```

    NODE_ADD,      // 加法节点
    NODE_SUB,      // 减法节点
    NODE_MUL,      // 乘法节点
    NODE_DIV       // 除法节点
} NodeType;

// 定义语法树节点
typedef struct TreeNode {
    NodeType type;
    int value;      // 数字值 (仅对 NODE_NUM 有效)
    struct TreeNode* left;  // 左子树
    struct TreeNode* right; // 右子树
} TreeNode;

// 全局变量: 当前读取位置
char* current;

// 函数声明
TreeNode* parseE();
TreeNode* parseEPrime(TreeNode* left);
TreeNode* parseT();
TreeNode* parseTPrime(TreeNode* left);
TreeNode* parseF();
TreeNode* createNode(NodeType type, int value, TreeNode* left, TreeNode* right);
void skipSpaces();

// 跳过空格
void skipSpaces() {
    while (*current == ' ' || *current == '\t') {
        current++;
    }
}

// 创建新节点
TreeNode* createNode(NodeType type, int value, TreeNode* left, TreeNode* right) {
    TreeNode* node = (TreeNode*)malloc(sizeof(TreeNode));
    node->type = type;
    node->value = value;
    node->left = left;
    node->right = right;
    return node;
}

// 解析 E -> T E'
TreeNode* parseE() {
    TreeNode* t = parseT();
    return parseEPrime(t);
}

// 解析 E' -> + T E' | - T E' | ε
TreeNode* parseEPrime(TreeNode* left) {
    skipSpaces();

    if (*current == '+') {
        current++;
        TreeNode* t = parseT();
        TreeNode* node = createNode(NODE_ADD, 0, left, t);
        return parseEPrime(node);
    } else if (*current == '-') {
        current++;
        TreeNode* t = parseT();
        TreeNode* node = createNode(NODE_SUB, 0, left, t);
        return parseEPrime(node);
    }

    // ε production
    return left;
}

// 解析 T -> F T'
TreeNode* parseT() {
    TreeNode* f = parseF();
    return parseTPrime(f);
}

// 解析 T' -> * F T' | / F T' | ε
TreeNode* parseTPrime(TreeNode* left) {
    skipSpaces();

    if (*current == '*') {
        current++;
        TreeNode* f = parseF();

```

```

        TreeNode* node = createNode(NODE_MUL, 0, left, f);
        return parseTPrime(node);
    } else if (*current == '/') {
        current++;
        TreeNode* f = parseF();
        TreeNode* node = createNode(NODE_DIV, 0, left, f);
        return parseTPrime(node);
    }

    // ε production
    return left;
}

// 解析 F -> (E) | n
TreeNode* parseF() {
    skipSpaces();

    if (*current == '(') {
        current++; // consume '('
        TreeNode* e = parseE();
        skipSpaces();
        if (*current == ')') {
            current++; // consume ')'
        } else {
            fprintf(stderr, "Error: expected ')\n");
            exit(1);
        }
        return e;
    } else if (isdigit(*current)) {
        int value = *current - '0';
        current++;
        return createNode(NODE_NUM, value, NULL, NULL);
    } else {
        fprintf(stderr, "Error: unexpected character '%c'\n", *current);
        exit(1);
    }
}

// 主解析函数
TreeNode* parse(char* exp) {
    current = exp;
    TreeNode* tree = parseE();
    skipSpaces();
    if (*current != '\0') {
        fprintf(stderr, "Error: unexpected characters after expression\n");
        exit(1);
    }
    return tree;
}

// 辅助函数: 打印语法树 (中序遍历)
void printTree(TreeNode* node) {
    if (node == NULL) return;

    if (node->type == NODE_NUM) {
        printf("%d", node->value);
    } else {
        printf("(");
        printTree(node->left);
        switch (node->type) {
            case NODE_ADD: printf(" + "); break;
            case NODE_SUB: printf(" - "); break;
            case NODE_MUL: printf(" * "); break;
            case NODE_DIV: printf(" / "); break;
            default: break;
        }
        printTree(node->right);
        printf(")");
    }
}

// 辅助函数: 释放语法树
void freeTree(TreeNode* node) {
    if (node == NULL) return;
    freeTree(node->left);
    freeTree(node->right);
    free(node);
}

// 测试主函数
int main() {
    char expr[] = "3+4*2/(1-5)";

```

```

printf("Expression: %s\n", expr);

TreeNode* tree = parse(expr);
printf("Parse tree: ");
printTree(tree);
printf("\n");

freeTree(tree);
return 0;
}

```

说明：

- (1) **数据结构**：TreeNode 包含节点类型、值（对于数字节点）、左右子树指针。
- (2) **递归下降**：每个非终结符对应一个解析函数：
 - `parseE()`：解析 $E \rightarrow TE'$
 - `parseEPrime(left)`：解析 $E' \rightarrow +TE' \mid -TE' \mid \varepsilon$ ，参数 `left` 是左操作数
 - `parseT()`：解析 $T \rightarrow FT'$
 - `parseTPrime(left)`：解析 $T' \rightarrow *FT' \mid /FT' \mid \varepsilon$
 - `parseF()`：解析 $F \rightarrow (E) \mid n$
- (3) **左结合性**：通过 E' 和 T' 的实现，将左操作数传递下去并构建左结合的树。
- (4) **优先级**：通过文法的层次结构自然实现：乘除运算在 T 层，加减运算在 E 层，因此乘除优先级更高。